

Great question 👍

In **Object-Oriented Programming (OOP)**, **inheritance** is a mechanism where one class (child/derived) can acquire the properties and behaviors (fields and methods) of another class (parent/base).

In **Java**, inheritance supports **code reusability** and **method overriding**.

✓ Types of Inheritance

1. Single Inheritance

- One class inherits from another class.
- Example: class B extends A
- **Diagram:**
- $A \rightarrow B$

```
2. class Animal {
3.     void eat() {
4.         System.out.println("Eating...");
5.     }
6. }
7.
8. class Dog extends Animal {
9.     void bark() {
10.        System.out.println("Barking...");
11.    }
12. }
13.
14. public class Test {
15.     public static void main(String[] args) {
16.         Dog d = new Dog();
17.         d.eat();
18.         d.bark();
19.     }
20. }
```

2. Multilevel Inheritance

- A class inherits from a class, which itself inherits from another class.
- **Diagram:**
- $A \rightarrow B \rightarrow C$

```

3. class Animal {
4.     void eat() { System.out.println("Eating..."); }
5. }
6.
7. class Dog extends Animal {
8.     void bark() { System.out.println("Barking..."); }
9. }
10.
11. class Puppy extends Dog {
12.     void weep() { System.out.println("Weeping..."); }
13. }
14.
15. public class Test {
16.     public static void main(String[] args) {
17.         Puppy p = new Puppy();
18.         p.eat();
19.         p.bark();
20.         p.weep();
21.     }
22. }

```

3. Hierarchical Inheritance

- Multiple classes inherit from a single parent class.
- **Diagram:**
- A
- / \
- B C

```

4. class Animal {

```

```

5.     void eat() { System.out.println("Eating..."); }
6. }
7.
8. class Dog extends Animal {
9.     void bark() { System.out.println("Barking..."); }
10. }
11.
12. class Cat extends Animal {
13.     void meow() { System.out.println("Meowing..."); }
14. }
15.
16. public class Test {
17.     public static void main(String[] args) {
18.         Dog d = new Dog();
19.         d.eat();
20.         d.bark();
21.
22.         Cat c = new Cat();
23.         c.eat();
24.         c.meow();
25.     }
26. }

```

4. Multiple Inheritance (⚠️ Not supported directly in Java with classes)

- A class inherits from **two or more classes**.
- **Problem:** Leads to **ambiguity** (Diamond problem).
- **In Java:** Achieved using **interfaces**, not classes.
- **Diagram:**
- A B
- \ /
- C

```

5. interface A {
6.     void show();
7. }
8.
9. interface B {
10.    void show();
11. }
12.
13. class C implements A, B {
14.    public void show() {
15.        System.out.println("Multiple inheritance via interfaces");
16.    }
17. }
18.
19. public class Test {
20.    public static void main(String[] args) {
21.        C obj = new C();
22.        obj.show();
23.    }
24. }

```

5. Hybrid Inheritance

- Combination of two or more types of inheritance.
- Java **doesn't support hybrid inheritance** directly (to avoid complexity and ambiguity), but it can be achieved using **classes + interfaces**.

Example (Hierarchical + Multiple):

```

interface A {
    void show();
}

```

```

class B {

```

```

void display() {
    System.out.println("Class B");
}
}

```

```

class C extends B implements A {
    public void show() {
        System.out.println("Interface A implemented");
    }
}

```

```

public class Test {
    public static void main(String[] args) {
        C obj = new C();
        obj.display();
        obj.show();
    }
}

```

Summary Table

Type	Supported in Java	Example
Single	✅ Yes	One parent, one child
Multilevel	✅ Yes	Grandparent → Parent → Child
Hierarchical	✅ Yes	One parent, many children
Multiple (via class)	❌ No	Ambiguity issue
Multiple (via interface)	✅ Yes	implements keyword
Hybrid	Partially (with interface)	Mix of above

 Do you want me to also draw the **diagrams for each inheritance type** so it's super clear visually?

